Draft version November 17, 2025
Typeset using LaTeX **twocolumn** style in AASTeX7

Recovering Transfer Functions From Reverberation Mapping—a (De)Convolutional Neural Network Approach

Kirk Long,[1] Keith Horne,[2] Jason Dexter,[1] and Benoit Trembay[3]

[1] *JILA, University of Colorado Boulder*
[2] *University of St Andrews*
[3] *Environment and Climate Change Canada*

Submitted to ApJ

ABSTRACT

One of the hallmarks of active galactic nuclei are that they are highly variable with time. In watching the spectra vary it has been observed that the emission lines often appear to "reverberate"—that is they vary in response to continuum variations assumed to originate close to the black hole. This critical observation underlies the reverberation mapping technique, an elegant physics experiment that has allowed us to characterize the environment around many supermassive black holes in nearby active galactic nuclei. Recent observations are of such quality that the response can be measured as a function of velocity across the emission line, and in doing so we can construct velocity-delay maps that show the structure and physics of the gas in the broad-line region better than any other measurement to date. Unfortunately constructing such maps requires a deconvolution, and given that the data are often noisy and with gaps such deconvolutions are non-trivial. Here we present a novel deconvolution method for the recovery of velocity-delay maps using a custom convolutional neural network architecture, showcasing that such methods have great promise for the deconvolution of reverberation mapping data products. While we have designed this new method with the BLR in mind, in principle this technique could be applied to any reverberation deconvolution problem, including in the accretion disk and torus.

Keywords: Reverberation mapping, AGN, BLR, machine learning, neural networks

1. INTRODUCTION

Variability is an observational hallmark of quasars and active galactic nuclei that has been exploited to infer many underlying physical processes in these systems that are otherwise not directly observable. Reverberation mapping is one such technique, where variations in one part of the spectrum are assumed to drive variations in another part—for example the AGN broad-lines are assumed to reprocess light from the continuum (R. D. Blandford & C. F. McKee 1982). The inner components of most AGN are too small to resolve visually with current instruments, but vary on scales short enough to allow for the reverberation mapping technique to provide insights into the structure and kinematics of regions where sources are unresolved, usually at broad-line region scales and smaller (E. M. Cackett et al. 2021; B. M. Peterson 1993). Even in sources whose BLRs are marginally resolved by advanced instruments such as GRAVITY (GRAVITY Collaboration et al. 2019; GRAVITY+ Collaboration et al. 2023), reverberation mapping data play a critical role in constraining what geometries and kinematics are possible for the BLR (K. Long & J. Dexter 2025; G. A. Kriss et al. 2019; K. Horne et al. 2021; P. R. Williams et al. 2020) and may even enable the BLR to test the Hubble constant (Z.-X. Zhang et al. 2019).

In this paper we will focus on reverberation mapping as applied to the BLR, but note that the methods here are quite general and flexible and may be applied to other reverberation mapping problems. Originally reverberation mapping experiments in the BLR were 1D only (i.e. see K. T. Korista et al. 1995, for older reverberation results in NGC 5548), that is they only measured the entire emission line compared to the continuum. Advances in instrument and data processing techniques now allow for 2D reverberation mapping,

Corresponding author: Kirk Long
kirk.long@colorado.edu

where information can be obtained as a function of wavelength/velocity across the emission line (W. F. Welsh & K. Horne 1991; K. Horne et al. 2021).

Formally, reverberation mapping assumes variations in emission from the gas in the broad-line region are thought to be related to variations in the continuum via a convolution:

$$\Delta L(\nu,t) = \int_0^t \Psi(\nu,\tau)\Delta C(t-\tau)\mathrm{d}\tau = \Psi * \Delta C \quad (1)$$

Here $\Psi(\nu,\tau)$ is the 2D *frequency* and *time* dependent *response* function, which encodes how changes in the emission line $\Delta L(\nu,t)$ "respond" to changes in the continuum ΔC. Thus nearly all of the interesting physics that govern the BLR are contained within Ψ. Much work has gone into developing deconvolution methods for reverberation mapping, where the data is often sparse, messy, and may not strictly follow a linear convolution at all times (K. Horne et al. 1991; K. Horne 1994; J. H. Krolik & C. Done 1995). Unfortunately, different analytic methods for recovering Ψ from the data alone do not produce the same results, with simulated model data leading to erroneous recoveries (S. W. Mangham et al. 2019; J. H. Krolik & C. Done 1995). One can also forward-model the BLR to obtain a more exact response function than data-based methods, but at the cost of imposing a physical model that we do not know to be true P. R. Williams et al. (2020); A. Pancoast et al. (2014); K. Long & J. Dexter (2025).

While forward-modeling is a powerful tool for interpreting BLR observations, ideally we would be able to recover the response equally well without having to assume a particular model. If our data (and the system) were idealized and taken over long periods of time, one could simply solve for Ψ from ΔL and ΔC in Equation 1 using the convolution theorem:

$$\Psi = \mathcal{F}^{-1}\left(\frac{\mathcal{F}(\Delta L)}{\mathcal{F}(\Delta C)}\right) = \mathcal{F}^{-1}\left(\frac{1}{\mathcal{F}(\Delta C)}\right) * \Delta L \quad (2)$$

$$= \boxed{f(\Delta C) * \Delta L} \quad (3)$$

where $\mathcal{F}$ denotes the Fourier transform and $\mathcal{F}^{-1}$ its inverse. In practice measurement problems prevent this, but fortunately many analytic extensions have been developed that allow approximate deconvolutions of real data.

As shown in the boxed form of Equation 2, an equivalent way of thinking about the recovery of Ψ is that we are looking to find some complicated function $f(\Delta C)$ that we can convolve with ΔL. A CNN is designed precisely with this task in mind: the network essentially "learns" the complex and highly nonlinear function $f(\Delta C)$ as the optimized weights are convolved with the input emission line light curves to map back to the target response function. For example if we wanted to deconvolve a nearly perfect dataset but with some simple noise that we wanted to remove with a signal to noise ratio (SNR) cut we could write $f(\Delta C, \mathrm{errors})$ as the Wiener deconvolutional operator, i.e. $f(\Delta C, \mathrm{errors}) = \mathrm{f(C, SNR)} = \mathcal{F}^{-1}\left(\frac{1}{\mathcal{F}(\mathrm{C})}\left[\frac{|\mathcal{F}(\mathrm{C})|^2}{|\mathcal{F}(\mathrm{C})|^2+\mathrm{SNR}}\right]\right)$. Here our errors and data are too complicated to be inverted using an $f(\Delta C)$ as simple as the Wiener deconvolution, but the most flexible natural extension of this kind of idea is to use $f(C) = \mathrm{CNN(L)}$. This is the same general approach as any analytic deconvolution method but simply allows for the most flexible possible version of this type of operation. Thus the continuum and all of the errors both physical and systematic are absorbed into the parameters of the CNN, and in this approach what the CNN learns is what should be convolved with the input lightcurve to generate the correct transfer function. Because of the nature of the fundamental mathematical operation we are trying to approximate, we refer to this class of ML model as a (de)convolutional neural network ((D)CNN) throughout the rest of this paper.

In RM `MEMecho` is often used to estimate Ψ (K. Horne et al. 1991; K. Horne 1994), a code which essentially tries to maximize the entropy of any proposed transfer function that maps between the observed continuum and emission line lightcurves. This is a powerful and strongly motivated approach to solving this problem given the complications possible in the data, but it limited in its flexibility and the results can be influenced by the reference map used in fitting. The problem is complicated, and there is "evidence that certain line light curves cannot be fully described by the simple linear convolution model" (J. H. Krolik & C. Done 1995). This strongly motivates the work described here, in which we propose the use of a (D)CNN to recover Ψ. Using a (D)CNN method allows for much more robust flexibility and less human bias in obtaining Ψ but at the cost of interpretability—whereas in something like `MEMecho` it is clear what operations are happening the (D)CNN is somewhat of a black box, although we attempt to mitigate this interpretability problem in Section 8.

CNNs have long been successfully used for image deconvolution both in industry (D.-W. Li et al. 2021; C. Liu et al. 2022; L. Xu et al. 2014) and astronomy (J. Herbel et al. 2018; U. Akhaury et al. 2022), but an analogous approach has yet to be applied to the deconvolution of BLR RM data. This approach is strongly physically and mathematically motivated. Decades of RM mea-

surements have robustly shown that ΔC,ΔL, and Ψ are connected, but the precise relationship is muddled by various physical and systematic sources of noise. As shown in the boxed form of Equation 2, an equivalent way of thinking about the recovery of Ψ is that we are looking to find some complicated function of ΔC that we can convolve with ΔL. A CNN is designed precisely with this task in mind: the network essentially "learns" the complex and highly nonlinear function $f(\Delta C)$ as the optimized weights are convolved with the input emission line light curves to map back to the target response function. Another ML technique—variational autoencoders—has recently shown promise for recovering response functions for AGN coronae from X-ray lightcurves (S. Deesamutara et al. 2025), further motivating an ML application to the BLR RM inversion problem.

There are few, if any, interesting physics in the deconvolution itself, but failing to do it correctly prevents us from unraveling the true nature of the BLR. As the 2D lightcurves essentially represent a "blurry" image, we follow a hybrid approach for image deconvolution inspired by L. Xu et al. (2014) and K. He et al. (2015) by training a simple (D)CNN using Julia's `Flux.jl` framework (M. Innes et al. 2018; M. Innes 2018), and in this paper we will showcase the promise of this method for the deconvolution of real RM data.

2. SUMMARY OF MACHINE LEARNING MODEL ARCHITECTURE AND IMPLEMENTATION

We use Julia's `Flux.jl` framework for creating a custom CNN architecture to tackle this problem. In our testing we find that it is possible to robustly recover transfer functions of a variety of shapes with just a few simple layers:

1. We split the input into three independent paths, where each path has convolutional layers with filters corresponding to small, medium, and large velocity and temporal scales. This allows the model to learn information at each scale without being muddied by the other filters.

2. The output from these independent paths is then joined together and passed to a skip connection chain. Within the chain the model performs each convolutional filter in series going from small to large timescales and then small to large velocity scales. At the end the results are added to the unadultered output of the split/join chain before. This design choice is inspired by RESNET (K. He et al. 2015) and allows the model to recall previously learned information more easily, in this case information about each of the three possible scales we include independently.

3. To increase the depth of the neural network, we then perform the above sequence again, with the only difference being that the inputs to the split/join chain are now larger in the channel dimension as a result of the output from the skip connection chain.

4. Finally, we apply a $(1, 1)$ convolutional filter across the output from the second loop of items 1-2 and then output the model's prediction.

After each convolutional layer we regularize by including batch normalization (BN) and dropout (Drop) layers and then apply a rectified linear unit (ReLU) activation function, with the exception of the final output layer which includes only the ReLU activation function after taking the final convolution. A flowchart diagram of this model architecture is shown in Figure 1.

To train the model we use a synthetic AGN lightcurve generated according to a damped random walk (S. Kozłowski et al. 2010; C. L. MacLeod et al. 2010) that is convolved with a few thousand possible model transfer functions to generate synthetic output lightcurves. While damped random walks are likely an oversimplification of the true variability of quasars (V. P. Kasliwal et al. 2015; W. Yu et al. 2022), they are fast and easy to compute, and really the exact shape of the driving lightcurve does not particularly matter as we are tasking the models with learning in general how to perform the deconvolution. These synthetic output lightcurves are then subject to random observational noise and data dropouts meant to mimic bad data and/or observational gaps, but they are assumed to have been properly detrended as there are potentially interesting physics that motivate the de-trending process that we do not want the (D)CNN to absorb (K. Horne et al. 2021). The transfer functions used to generate the lightcurves are drawn from three populations:

1. A distribution of Ψ that come from a selection of "basis" shapes, including rings, guassians, and exponential decay profiles in velocity-delay space.

2. A distribution of Ψ drawn from an analytic Ψ for an accretion disk. While the accretion disk signal likely does not extend to the BLR, we include this as it is another nice representative "picture" of how gas can respond to changes in the continuum, and in the hopes that the model may be able to learn to disentangle contributions from the accretion disk that may contaminate BLR measurements and vice versa.

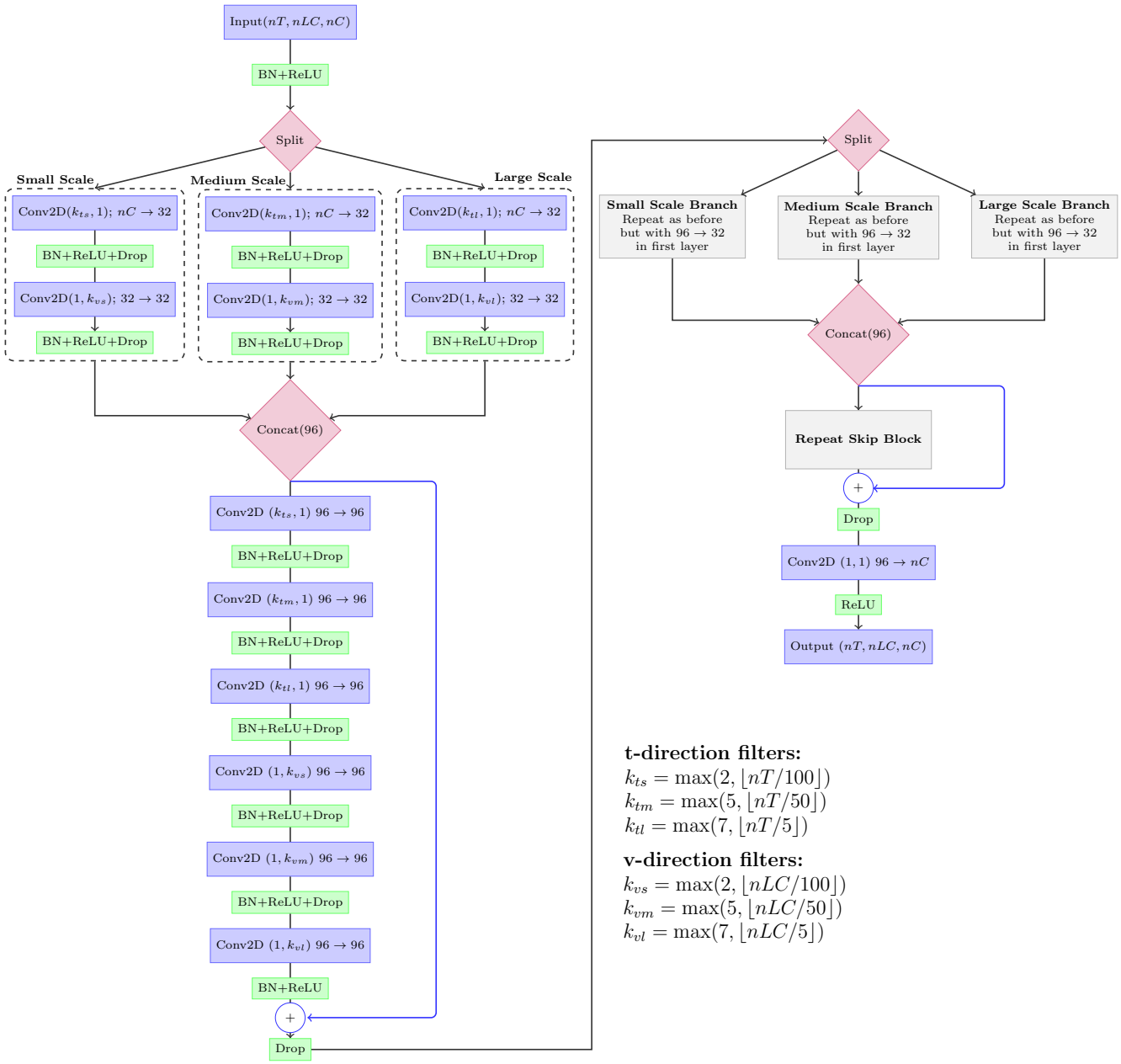


Figure 1. A schematic of the general architecture of our (D)CNN. The indigo boxes represent the input, output, and convolutional layers, with each convolutional layer including a description of the number of channels at that step (we fix $nC = 1$ in this work). The green boxes represent regularization—BatchNorm (BN) and Dropout (Drop)—and activation—ReLU—layers that occur after each convolutional layer. The red diamonds represent split/join operations, where in the case of joining the number of channels after joining is denoted in parentheses. The left panel shows one split/join and skip connection chain in full, while the right panel showcases that our model architecture essentially performs these operations twice. Note that while the full 2D schematic is shown in the 1D case the general pattern is the same but the convolutions are 1D and only along the time dimension thus there are $\sim$ half the number of layers than in the 2D case. When the number of velocity channels is increased the gradients become large in the 2D case, and thus sometimes memory requirements necessitate using only one split/concat loop instead of both.

3. A distribution of Ψ drawn from simple kinematic disk, cloud, and combined models of the BLR similar to those described in K. Long et al. (2023) and K. Long & J. Dexter (2025).

In addition to generating samples independently, all three of these sources of Ψ are also randomly added in combination to each other to generate additional samples, such that one lightcurve could be generated from say a basis shape alone or a basis shape in combination with a kinematic BLR model or a basis shape in combination with both a kinematic disk model and an analytic accretion disk model. The training set is generated such that the distribution of each class is equal (1/2 each class independently + 1/2 combinations of the classes). All the resulting transfer functions are normalized, as we are asking the model to predict their shapes and not their intensities relative to each other. The full training set is too large to cheaply host on the internet at this time, but is available on request from the author.

We then train an ensemble of ~ 100 (D)CNNs on our fake data, splitting it into 70% training and 30% testing data subsets. Each model is initialized with random Glorot normal (X. Glorot & Y. Bengio 2010) weights and is trained using the Adam optimizer with a learning rate of $\sim 10^{-4}$, a value found experimentally to produce good results. We use a simple mean square error loss function to evaluate each model during training, which is evaluated with gradient descent optimization until each model arrives at some local minimum in the loss function. Each model is trained for a variable amount of epochs, with the training automatically stopping when one of the following conditions is reached:

1. We compare the average training loss of the last five epochs with the average loss of the five before that. If there is no change or the the average is increasing we stop training.

2. We perform a similar test for the loss on the validation function and similarly stop training if the average validation loss across the five window average flattens out or increases, as this indicates the model may be overfitting to the training data. Note that although we evaluate the performance of the loss function on the validation data the model never sees this data in each gradient descent update, thus the training is blind to the validation set except in this autostopping routine.

During training we create checkpoints of our model state every time the validation loss reaches a previously unseen low, and reverts to the last saved checkpoint in case training is stopped before the maximum number of epochs is reached. The choice of using the previous 10 epochs in this way for the autostop criteria is arbitrary and was chosen experimentally as it provided a good balance of allowing enough chances for the model to "escape" a local minima it may have stalled out in versus wasting computational resources when it truly has found the best solution for its initial conditions and optimization rate. In general we find that each model trains for ~ 50 epochs before this autostop criteria is usually triggered.

After each model in our ensemble has been trained we generate a distribution of their final validation losses and perform a 1σ cut to keep only the best performing models. This cut is again chosen arbitrarily but is done to remove models which did not learn very much about the problem due to their initial conditions and/or learning rates as well as to penalize models which overfit the data without generalizing what they learned to the broader scope of the problem.

We can then generate a prediction and confidence interval using the remaining models in the ensemble, where the prediction is the mean of each individual model's prediction and the confidence interval the standard deviation of the individual model predictions. Constraining uncertainties with machine learning methods is notoriously difficult, but we believe this ensemble approach, while computationally more expensive than training a single model, is robust and allows for a real estimate of the uncertainty on each prediction in addition to allowing the models to explore the various minima that may exist in the loss function as there is no guarantee that there is a global minimum or that each model will converge to the same minimum. The simplicity of the model architecture allows for each model to be trained with relatively few resources—the individual model sizes are $\lesssim 30$ MB and many of the results shown here were generated on a laptop with an NVIDIA 5070 Ti GPU with just ~ 6000 CUDA cores. The most computationally expensive models are the 2D cases shown here as the gradient sizes become larger, but even these are easy to train on a single node of a high-performance computing cluster with a single A100 GPU.

While the process outlined above allows the (D)CNN to learn the specifics of one continuum lightcurve through training, it is easy to apply the lessons learned to other possible continuum lightcurves through transfer learning. To do this we generate a new training dataset in the same way as before but with a new continuum lightcurve, then we train the ensemble of models again but this time instead of initializing their weights randomly we start with their previously learned parameters from the old continuum. In this way the models are al-

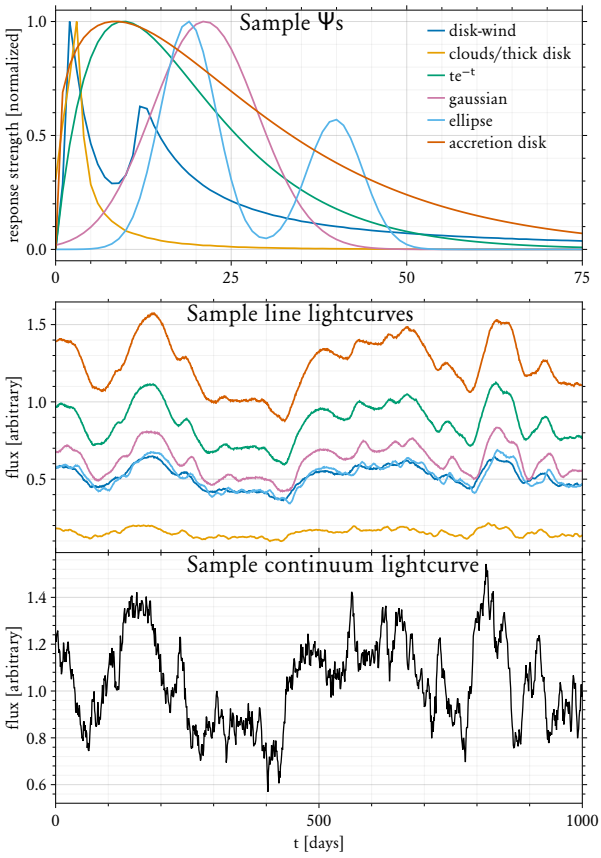


Figure 2. Top: Sample 1D transfer functions for each "basis" function used in training the model. **Middle**: Sample lightcurves resulting from the convolution of the corresponding transfer function from the top panel with the continuum. **Bottom**: The DRW continuum lightcurve convoled with the top panel to generate the middle panel.

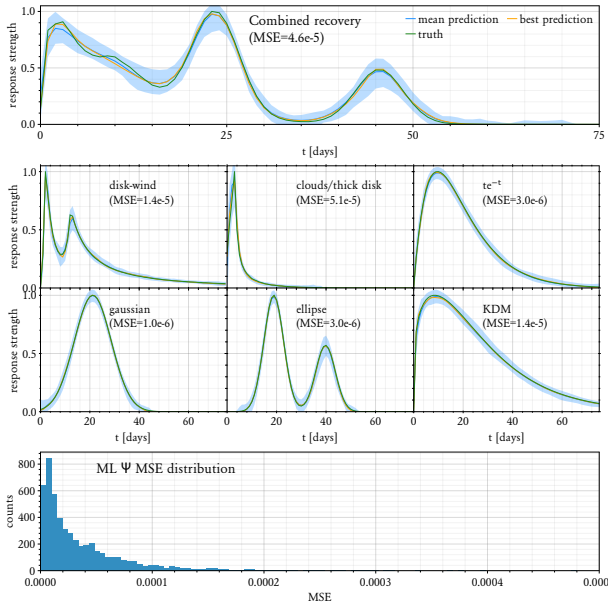


Figure 3. Top: Sample recovery of a more complicated transfer function formed from the combination of several of the basis transfer functions shown in Figure 2. **Middle**: Sample recoveries of each of the six "basis" transfer functions. **Bottom**: Distribution of errors for model predictions.

ready much closer to the solution than they are when initialized randomly, and in our tests they take only a few epochs of training to transfer what they learned from the old continuum to the new one, thus updating the model for a new continuum is a relatively quick and easy task when compared with the initial fits. In this way it is better to think of this method not as a general catch-all approach for every possible source and transfer function, but instead as a novel fitting method for each particular source that can interpolate in a highly non-linear way between the supplied transfer functions it has seen in training to generate a flexible best fit.

3. 1D TEST CASES

First, we demonstrate the recovery of the one-dimensional transfer function $\Psi(\tau) = \int \Psi(v, \tau)\mathrm{d}v$. In this case we follow the approach outlined in Section 2 but we include only information in the delay dimension as any velocity-dependent effects are integrated out. A sample transfer function and line lightcurve from each possible distribution included in our training set is shown in Figure 2—in total we generate $\sim$ 15,000 fake samples drawn from these distributions and split the resulting dataset 70/30, such that the model sees roughly 10,000 total samples in training and we can then test its predictions on the remaining 5,000 samples it is blind to (the validation set). In this scenario we assume a continuum and integrated emission line lightcurve have been measured at a fixed cadence for $\sim$ 1000 times the step size, which we scale in our plots to take a fixed cadence of one day with a total time baseline of 1000 days. Because of this choice we do not allow any transfer functions shown to the model in training to have any significant response past $\sim$ 1/10th of the total time baseline ($\sim$ 100 days). This ensures the model only picks up on trends that should be possible to measure given the constraints of the observations.

We start by training an ensemble of $\sim$100 (D)CNNs as described in Section 2, and after training and imposing our cut to keep only the best performing models the final ensemble includes $\sim$ 30 trained (D)CNNs. The top panel of Figure 3 shows sample model recoveries for each distribution in the dataset, while the bottom panel of shows the distribution of errors for all of the model predictions.

Note that the best fit and mean fit lines are quite close to each other, indicating that all of the models in

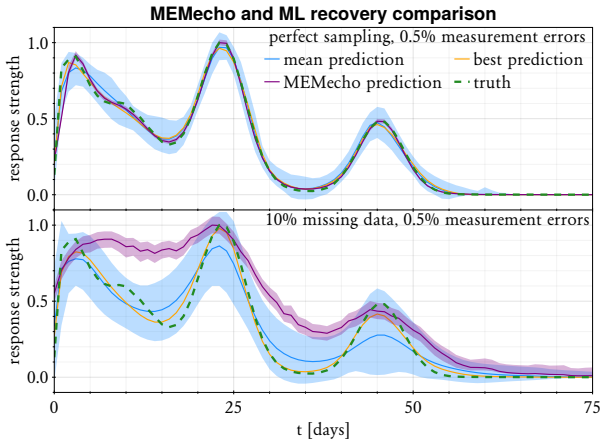


Figure 4. Comparing how the novel (D)CNN and analytic `MEMecho` techniques performs when the training data quality are degraded **Top**: The reference predictions for the more complicated transfer function shown in top panel of Figure 2 trained on perfectly sampled data with small measurement errors. **Bottom**: Sample recoveries of the same transfer function, but this time from data with random observational gaps.

the ensemble are learning generalizable information and it is not just one model that happens to luckily find the optimal local minimum that matches the data.

4. MISSING/ERRONEOUS DATA

While the results above already include noisy observations, they are perfectly sampled with a set cadence in time space. An important question is then how the model predictions might degrade given gaps in observations, and to test this question we train the models as before but this time remove 10% of all the time samples. As expected, this does degrade the quality of the predictions somewhat, particularly in resolving sharp features, but overall the models still learn an impressive amount of generalizable information and recover the transfer functions well. For example, when the input lightcurves are just noisy but perfectly sampled as shown in Section 3 the average MSE is $\sim 5 \times 10^{-5}$, and when sampling gaps are introduced this average error increases to $\sim 1.6 \times 10^{-4}$, a factor of ~ 3 increase, a ratio that is similar in both 1D and 2D inversion tests. In Figure 4 we showcase how degrading the training data quality affects the recovered 1D transfer functions. As expected the recoveries are noisier and less precise, but it clearly can still recover overall trends, and importantly in both the perfect and missing data cases the error bar always encapsulates the true response function.

This inversion problem has existed long before machine-learning methods were a tractable possible solution, and the `MEMecho` K. Horne et al. (1991); K. Horne (1994) code has existed as the most widely-used analytic approach to tackling this inversion problem. Thus in Figure 4 we also show for comparison the `MEMecho` inversions for the same data. As Figure 4 shows, the `MEMecho` inversion is nearly perfect when given nearly perfect data as expected, but in degrading the data quality the `MEMecho` fit begins to diverge from the ground truth, while the mean ML prediction remains relatively consistent between the top and bottom panels just with significantly wider error bars. The error bars on the (D)CNN model predictions are generated by having each of the models in the ensemble generate 100 predictions from 100 distinct lightcurves, for a total of 10^4 samples, which we then calculate a mean, best fit, and upper and lower standard deviations on that mean to show in the ribbon. To generate the error bars on the `MEMecho` fits we ran `MEMecho` 10^4 times on fake data generated in the same way as it was for the ML model predictions, with the ribbon corresponding again upper and lower standard deviations on the mean `MEMecho` fit.

This is one advantage of the (D)CNN methodology—when training an ensemble of models it is easy and cheap to generate uncertainties directly, whereas while an individual run of `MEMecho` is relatively fast generating uncertainties is much slower as the algorithm must be run many times over. It is encouraging that the (D)CNN method error bars look realistic in that the models are more unsure when the data quality is worse, and the places it is most unsure are where there are sharp features in the data. Importantly that in both cases the ground truth is always captured by the (D)CNN error bars, which is not always true for `MEMecho` due to the analytic nature of its setup, which preferentially steers it towards similar solutions. It is also encouraging that the best and mean predictions for the (D)CNN method are very similar, showcasing that all of the models are learning quite generally.

5. 2D TEST CASES

While the 1D results are encouraging, modern instrumentation and data analysis techniques have enabled two dimensional reverberation mapping campaigns, where the lightcurves are measured as a function of velocity across the emission line. Such 2D datasets provide new opportunities to constrain the fundamental nature of the broad-line region and reveal important information that is missed in recovering just the integrated 1D response (K. Long et al. 2023). Obtaining the most accurate full 2D echo maps is thus critical to unraveling the true nature of the broad-line region.

Here we demonstrate that our (D)CNN technique works for the full 2D case as well, performing similar

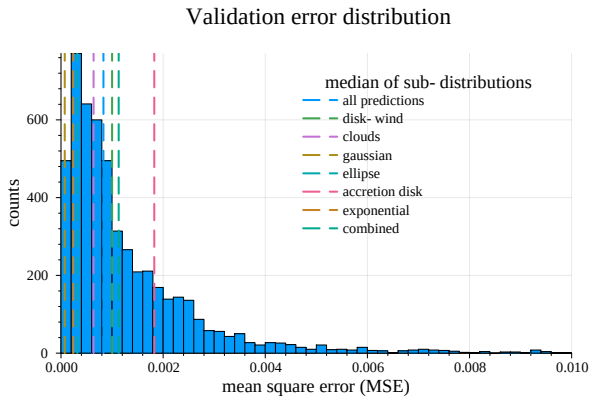


Figure 5. The distribution of errors for the sample ensemble of 2D models. Full examples of each model transfer function and lightcurve inputs are in Figure 11 in the Appendix, and a sample inversion for a complicated hybrid BLR is shown in Figure 6. Vertical lines showcase the median error of each sub-population for each of the basis functions.

tests as shown in Section 3. We again train an ensemble of models following the approach outlined in Section 2, but this time use full 2D transfer functions to generate lightcurves at 25 distinct velocity bins, examples of which are shown in full in the Appendix. Unfortunately in training the gradients become much larger and more difficult to compute in the 2D case, so we restrict the model depths to just one split/skip layer as shown in Figure 1, which reduces the model sizes to just ~ 10 MB. To start, we again use data with measurement errors but full time-sampling. Given the complexity of the problem and to save space instead of showing sample recoveries for all basis functions and a complicated case we instead show just one showcasing a more complicated combined BLR as the 2D analogue for the top panel of Figure 3 (Figure 6). This more complicated transfer function is a combination of several possible basis functions, with the strongest components being a mostly "cloud" distribution at small times followed by a mostly "disk-wind" distribution at longer times. This creates a complicated structure in velocity-delay space that the model recovers qualitatively quite well.

The inversion shown in Figure 6 was selected as it was the closest "combined" prediction to the median error in the distribution shown in Figure 5, making it a good representation of the quality of possible predictions for this (D)CNN approach. Qualitatively the model does a good job in identifying where the distribution of gas lies around the black hole when comparing the top left and top middle panels showing the ground truth and model prediction, respectively. Where the model is led astray is clear in the middle left and middle center panels—in the residuals it is clear that the mean prediction gets the width wrong by a $\sim$ pixel, and because the edge is so sharp in the disk dominated portion of the ground truth this results in higher errors near these edges. While getting the overall morphology near line-center correct, the model also underpredicts the power in some of these cells, which is particularly evident in the bottom left/middle panel line profile residuals. The σ panel also shows this—the ensemble of models is most unsure of exactly where the edge of the ring/disk structure is and how much power should go near line center, which is an encouraging diagnostic as this means if we did not have access to the ground truth we would still have accurate information about which portions of the inversion were the least reliable. The 1D collapses in the bottom left/center and right top/middle panels show that the inversion quality has not decreased from the original 1D results presented in Section 3, despite having a much more complicated architecture through the introduction of the second dimension in the data quality. Note that here the uncertainty ribbons are underestimates, as this is just the uncertainty from the ensemble of 100 models applied to the same input lightcurve. This isolates the uncertainty in the (D)CNN models themselves, but in practice one would also feed many realizations of possible lightcurve inputs given their measurement errors which would increase these uncertainties.

As Figure 5 shows, in general the "shape" basis functions are easiest for the model to learn and the "physical" models the hardest, particularly in the case of the "disk" type inversions. This is likely due to the fact that finer structural changes in these classes of models lead to scenarios like that described above in the interpretation of Figure 6, but overall the distribution between the classes is relatively tight and each is predicted well.

The bottom right panel of Figure 6 verifies that our models do not overfit the data and our training procedure as outlined in Section 2 is working as intended. In particular it is encouraging to see that the morphology of the validation and training curves is nearly identical, which indicates that the models are learning strongly generalizable information about the inversion problem. Note that the relative offset between each curve is a byproduct of the fact that the training set contains more data (70% of the total) than the validation set (30% of the total).

6. TRANSFER LEARNING

All the results thus far have essentially been an exercise in model fitting, where the (D)CNN has adapted itself to fit one particular continuum lightcurve. While this is valuable alone, ideally our model could adapt to a variety of continuum lightcurves. While any particular

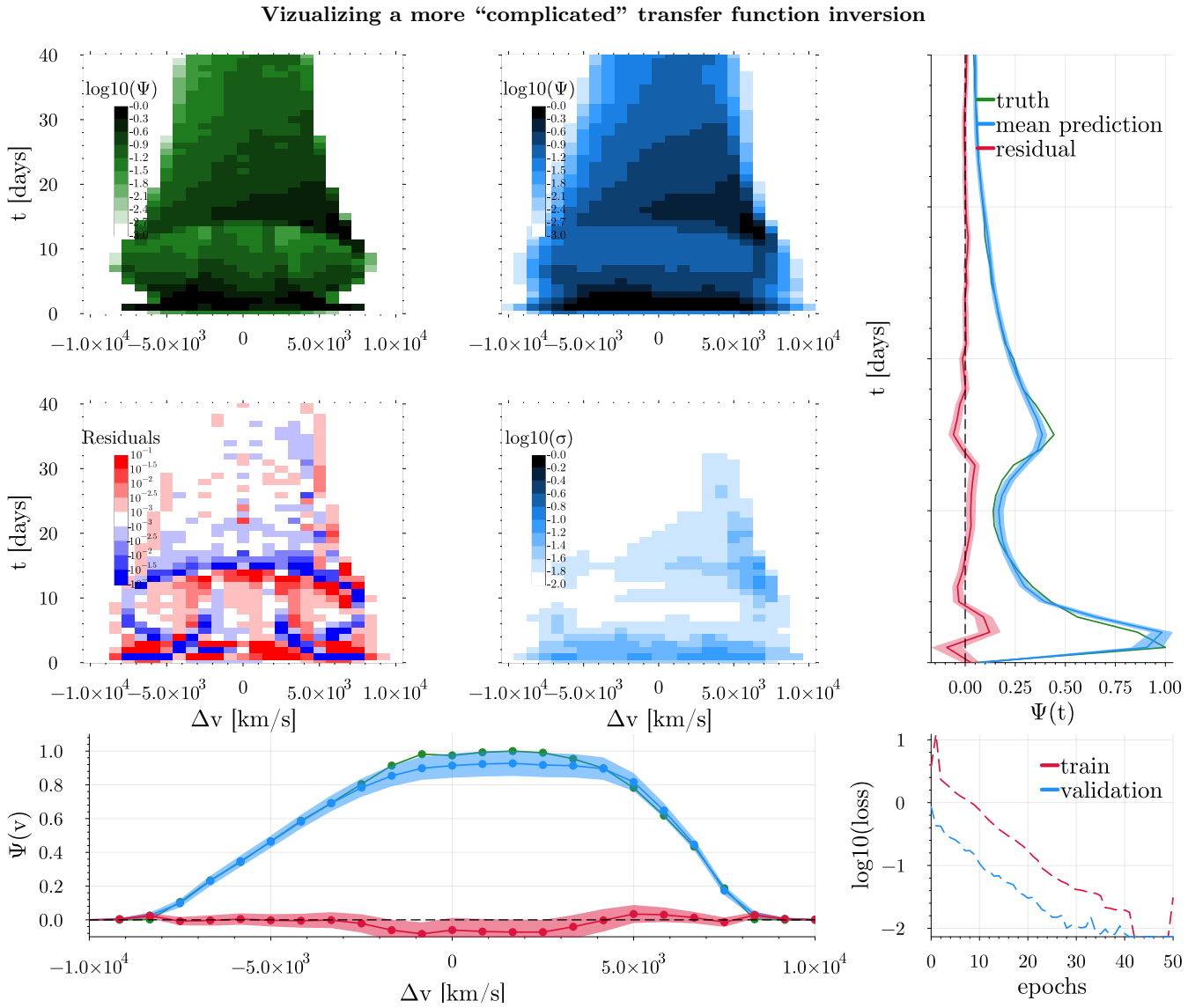


Figure 6. Top left: Sample full 2D transfer function of a complicated combined model with both a "disk-wind" and "cloud" component. **Top middle**: (D)CNN model recovery of the full 2D transfer function. **Middle left**: Residuals (prediction in top middle panel - ground truth shown in top left panel). The range of the colorbar shows -10% - 10% variations on an exponential scale from the maximum ground truth value. Regions where the model overpredicts are shown in red, underpredicted regions are shown in blue, and areas in white are where the model prediction is within ±0.1% of the maximum ground truth power. Clearly the Keplerian disk's ring-like structure's sharpness in the ground truth poses some problems for the model, but overall it does quite well. **Middle right**: The uncertainty in the ensemble of predictions for this lightcurve. Note that the ensemble of models are most uncertain of where the edges are and how much intensity should go in the inner region, as highlighted in the residuals. **Top/middle subpanel**: The 1D transfer function (2D maps integrated as a function of velocity, see Figure 3) showing the mean model prediction in blue (with ribbon corresponding to uncertainty), the ground truth in green, and the residuals in red. **Bottom left/middle**: The 1D line profile (2D maps integrated as a function of time) again showing the mean model prediction in blue (with ribbon corresponding to the uncertainty), the ground truth in green, and the residuals in red. **Bottom right**: Mean loss curves for the ensemble of models, with the validation shown in blue and the training set shown in red. Note that the models stop learning after $\sim$ 30-40 epochs.

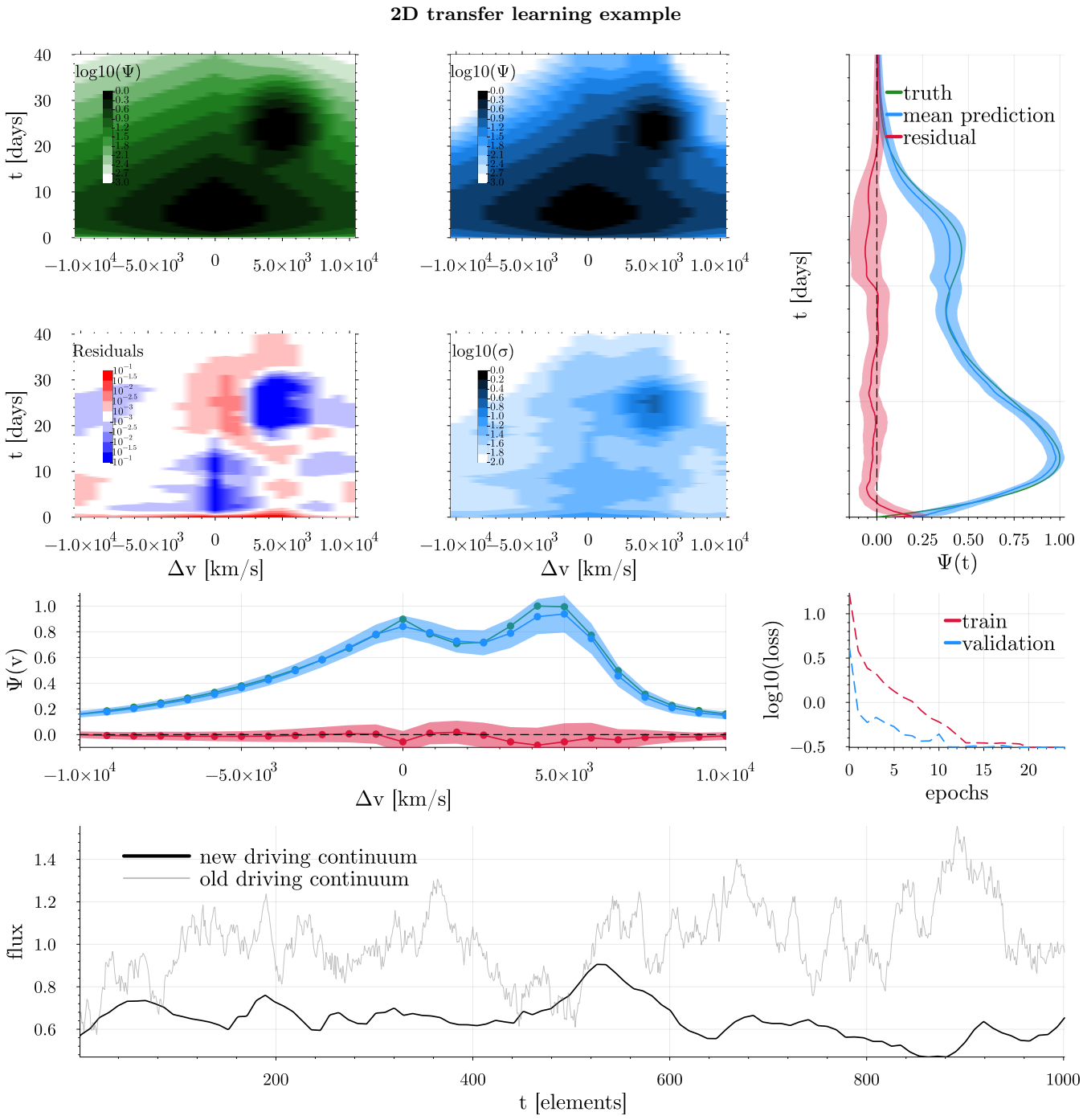


Figure 7. A 2D inversion of another complicated combined transfer function, this time the result of a two "shape" BLRs (an exponential decay component + a gaussian blob), demonstrating the general predictive power of (D)CNNs as essentially an "image" recovery method. The prediction was generated using an ensemble of models fine-tuned starting from the parameters used in generating the inversion shown in Figure 6 and with the same plot panels as described in the caption there. As shown in the bottom panel the continuum lightcurve and training datasets used for the fine-tuning were completely different in this case, with a significantly reduced physical timescale ($\sim$ 100 days instead of $\sim$ 1000 days) and more poorly resolved variations.

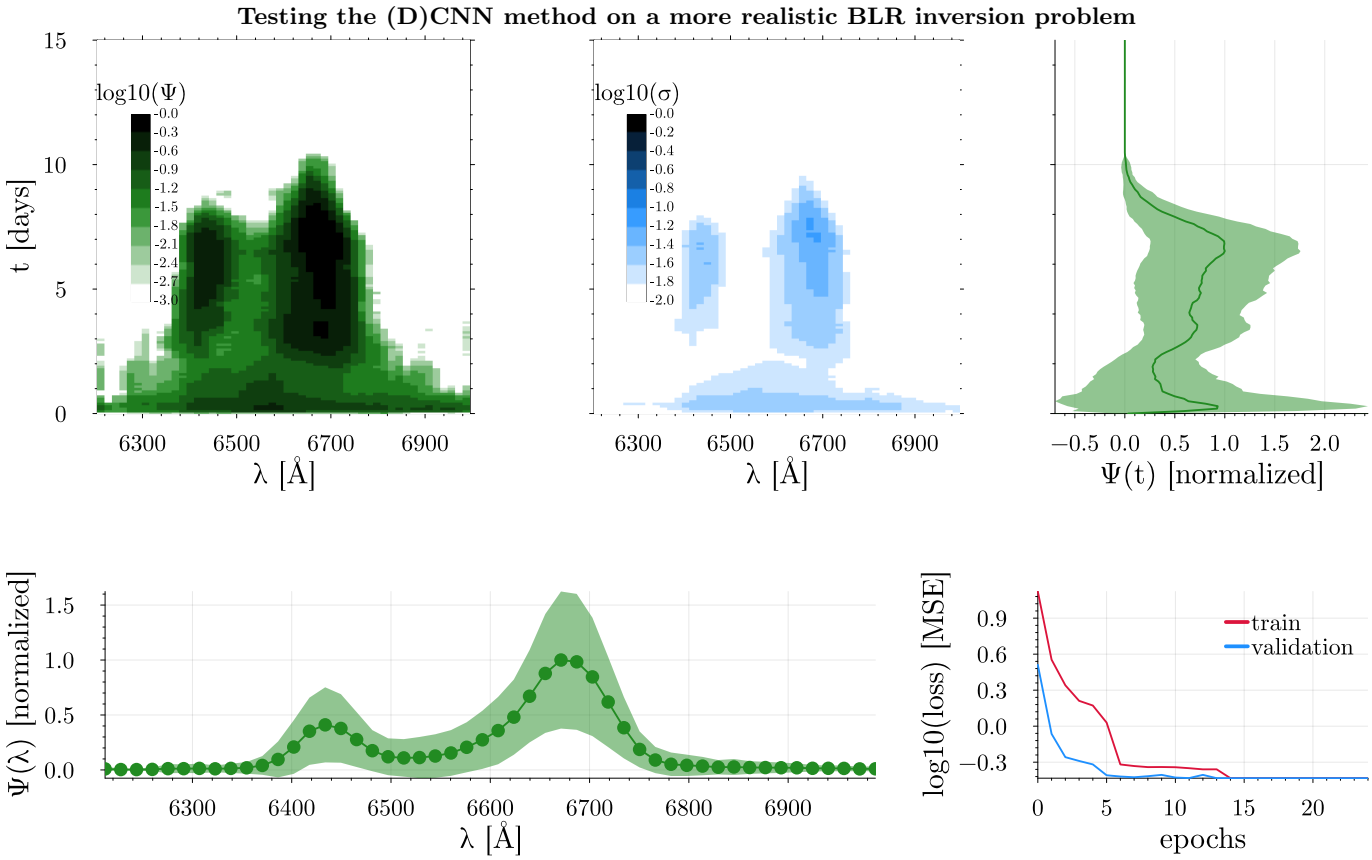
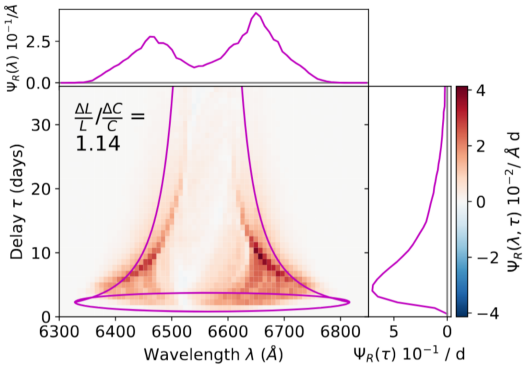


Figure 8. (D)CNN inversion of lightcurves extracted from a simulated quasar BLR dataset first presented in S. W. Mangham et al. (2019). **Top left:** mean predicted $\Psi(\lambda, t)$. **Top middle:** uncertainty on mean prediction. **Top right:** 1D transfer function $\Psi(t)$ with error bar corresponding to 1σ uncertainty. **Bottom left:** 1D $\Psi(v)$ with error bar corresponding to 1σ uncertainty. **Bottom right:** mean loss curves for the ensemble of models.

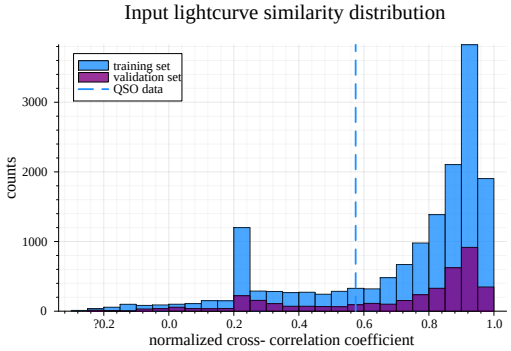
model at the end of training will be optimized to the continuum lightcurve used in generating its training samples, it turns out that it is quite easy to get the model to adapt to a different continuum lightcurve through transfer learning. This process essentially repeats the training process in a much smaller form, where instead of initializing the model weights randomly we start the model from its previous converged state and show it a new set of training samples generated with the new continuum lightcurve we want it to adapt to. In testing we have found that usually our models can adapt to a new continuum lightcurve in $\lesssim 10$ epochs, and it can do so with a much smaller training set (i.e. a few hundred samples instead of a few thousand).

The results of this process are demonstrated in Figure 7, where we took the parameters of the ensemble used to generate the inversions in Figure 6 and fine-tuned them on a new training data-set to generate the inversion shown for another complicated transfer function. Note that the loss drops rapidly and the training could have been stopped after just a few epochs to produce a similar quality inversion, showcasing how easy it is to transfer learn to new problems. The fake data used in training for this problem are generated using a much noisier continuum to simulate how one would apply this approach to real data where the intrinsic continuum is not so precisely known with distinctly shorter time scales in the transfer functions (note the y-axis scaling) to ensure the data is different than the sample it had trained on before. Additionally the data quality are degraded further as described in Section 4 by including gaps at 10% level similar to the 1D test shown in Figure 4. This process could be repeated by any end-user of our (D)CNN models to fine-tune to their needs—for example if one wanted to include a new basis function in the model training set one would simply generate the new samples for this basis function and then fine-tune the model on those new samples starting from its previously trained position as published here.

The goal of this transfer learning is to demonstrate how one might apply these models to real data, and to do this we perform a final test of our transfer-learned 2D (D)CNN models to see how well they can recover response functions from more realistic data. To ac-



(a) From S. W. Mangham et al. (2019), showing the true response of the quasar BLR model that the (D)CNN method tries to recover in Figure 8.



(b) Similarity distributions of the lightcurves used in training/validating the model, with the vertical line indicating how the set of extracted lightcurves for the quasar BLR in S. W. Mangham et al. (2019) compares to the distribution used in training.

Figure 9.

complish this we follow the "blind" test first performed with CARAMEL and `MEMecho` in S. W. Mangham et al. (2019). While the experiment has since been unblinded when S. W. Mangham et al. (2019) was published, we do not include anything like the full MHD + radiative transfer calculations that go into the more realistic BLR models shown in S. W. Mangham et al. (2019) in our training set, and we only use the series of synthetic spectra like one would from a telescope thus we follow the spirit of the blind test well. We extract continuum and line lightcurves from these synthetic spectra in the same way one would for use with `MEMecho`, and use the resulting continuum to generate the transfer learning dataset. We transfer learn 25 independent models on this dataset and keep the top 20 of them. We add noise to the line lightcurves extracted from the S. W. Mangham et al. (2019) spectra to generate a set of 50 independent noisy lightcurves to use as inputs for every model in our ensemble, yielding a total of 1000 predictions and allowing us to provide robust confidence intervals.

The results of this "blind test" are shown in Figure 8, which demonstrates that the (D)CNN method recovers the shape and distribution of power decently well for this test case. The true response function is shown in Figure 9—the (D)CNN correctly identifies the velocity and time locations of peak response and the red-blue asymmetry in the response, but predicts extra power at low times, underpredicts at later times, and does not resolve the features as sharply as they are present in the ground truth, although note that the prediction in Figure 8 is shown in log scale to highlight structure while the ground truth shown in Figure 9 is not. This excess power near 0 days is physically interesting as this is the result of not having entirely subtracted out the continuum signal properly from the line lightcurves, as the training set we use does not have any continuum contamination. In the future we could adjust the training set to be more realistic, including contamination from the continuum, and believe this would likely resolve this "problem". It also showcases that this method has promise outside of applications directly to the BLR, and could likely be applied to RM in the accretion disk (F. Pozo Nuñez et al. 2025) and the torus (A. K. Mandal et al. 2024).

Figure 9 also highlights another potential way to test how confident one should be in a (D)CNN prediction. As shown in the bottom panel of Figure 9, the set of input lightcurves used in generating the (D)CNN prediction are significantly outside the peak of the distribution for what the model was shown in training. This indicates another potential area for improvement as well as a providing a qualitative way to benchmark trust in predictions—if the models had seen many similar sets of lightcurves in training we may be more confident in its output prediction and vice versa, and additionally we may improve the performance of our models by finding a way to generate samples that are statistically more similar to the data we wish to predict.

7. INTERPRETABILITY

While the results above are encouraging demonstrations that the (D)CNN method can recover transfer functions from RM observations, one drawback of many machine learning methods is there interpretability. In designing the architecture of our CNN (see Figure 1) we attempted to let relevant physical scales inform our choices of filters and layers, in the hope that this might make the resulting predictions more interpetable. For example, one might expect that a lightcurve that varies at very short timescales would see strongest activations in the layers of the network with small timescale features, and conversely an input lightcurve that varied

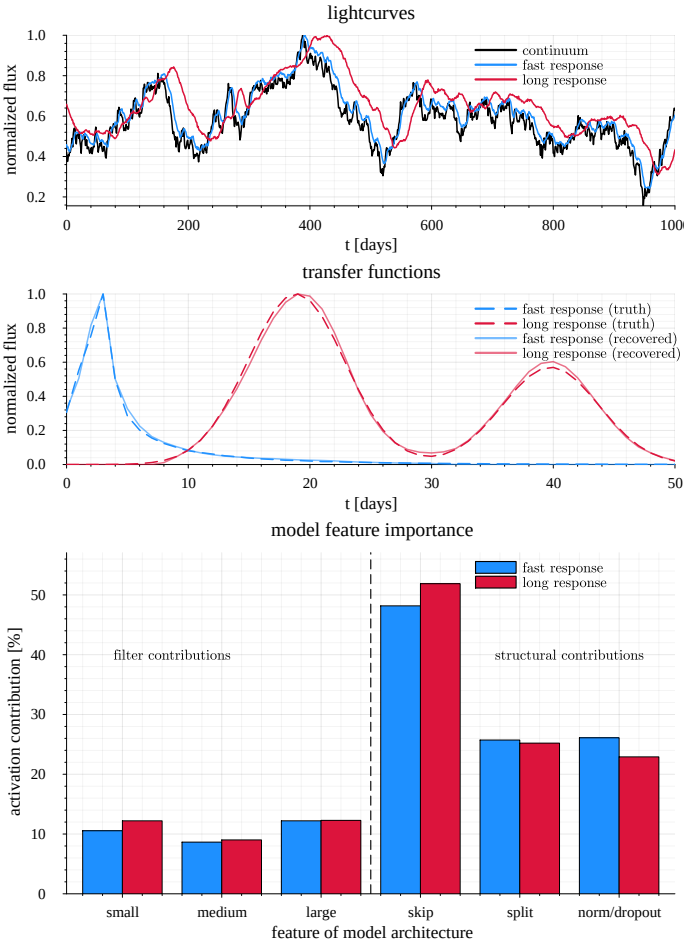


Figure 10. Here we showcase the percentage of activations (how strongly the model responds to an input and thus how much each feature contributes to the final prediction) for two very different input lightcurves. **Top:** The input lightcurves are shown in blue and red, while the continuum is shown for reference in black. **Middle:** The transfer functions used to generate the lightcurves in the top panel by convolving with the continuum. The blue is the "cloud" transfer function and the red is the elliptical transfer function as shown in Figure 3. The narrow peak at short delay times in the blue produces the fast variations seen in the lightcurve, while broader and later extent of the red transfer function results in the longer variations in its corresponding lightcurve. **Bottom:** The activation percentages of various components of our model architecture. The left three bars (small, medium, and large) correspond to the sum of all activations in each of the corresponding time filters. The blue "skip" bar showcases how important the overall skip connection design is for the model, while the purple "split" bar showcases the importance of the branching path components. Finally, the black "norm/droupout" bar demonstrates the importance of the normalization and dropout layers in making the prediction. Note that thus the right three bars add up to 100% but the left three bars do not as they are subcomponents of the "skip" and "split" paths.

slowly would see result in most of the model activations occuring in the layers that corresponded to the longest timescales. We test this idea in Figure 10, showing that this is distinctly *not* how the model operates. There appear to be no discernable changes between the activation of different timescale filters even when the inputs clearly show such different time-varying behavior, making interpreting how the (D)CNN works indeed more of a "black-box" than one might like. Interestingly the only trend between the two input lightcurves is the change in importance of the "skip" and "norm/dropout" layers, implying that the normalization and dropout layers appear to be more important for faster varying inputs.

8. DISCUSSION AND IMPLICATIONS

In this paper we have shown that a (D)CNN can successfully deconvolve reverberation mapping data products, both in 1D and in recovering the full 2D velocity-delay maps. The method works even in the face of significantly erroneous/missing data, and can naturally generate sensible confidence intervals on this prediction by when training an ensemble of models. The predictions generated with this new approach also encouragingly agree with the results of analytic inversion methods such as `MEMecho`. While we have designed this new method with the BLR in mind, in principle this technique could be applied to any reverberation deconvolution problem, including in the accretion disk and torus. Unfortunately even though the network's architecture was designed with physics in mind, simple tests to demonstrate the model behaves in the way we might intuitively expect show that its behavior is difficult to interpret.

Given the analysis in Section 7 it may make sense for future iterations of the model architecture to forgo the physically motivated "split" layers, and instead use only the "skip" connection architecture. This would likely save training time and make the models even more portable. Additionally in training future models one could explore different loss metrics—while here we have shown that the standard mean square error metric works well, custom loss functions incorporating further knowledge of the specific problem at hand may allow for further learning at the cost of computational complexity. For example one could imagine using a version of an image similarity metric such as the normalized cross-correlation coefficient—we plan to explore the effects of choice of loss function in future work. One could also envision tasking the (D)CNN with learning the uncertainty directly—i.e. learning a mean and standard deviation for each pixel—as opposed to estimating the uncertainty with an ensemble of models, but note that while this would save significant computational expense

the most reliable uncertainty estimate will always come from an ensemble approach due to the inherent nature of the local minima found via gradient descent each time the model is trained.

In favor of existing analytic methods like `MEMecho` are that such methods are interpretable, generating one-off predictions is faster than training a bespoke (D)CNN for any given problem, and it has been well-tested in many contexts and thus its blind spots are better known. In favor of the (D)CNN method presented here are that it is a more direct deconvolutional approach from a mathematical perspective, it is easier to characterize uncertainties by generating many samples very quickly once the model is trained, and it has fewer (or at least much more flexible) human biases in generating its predictions. In the future both approaches should be used in concert with one another as each has distinctive advantages and they are completely distinct methods for solving the inversion problem.

Obtaining the true 2D transfer function for a given source in this way is (physically) model agnostic, and in this way after the true 2D Ψ is recovered physical models can be compared with the resulting outputs to better constrain the fundamental nature of the BLR. There are many possible ways to match 1D observational products such as line profiles, delay profiles, and 1D response functions, and this degeneracy can only be fully broken by directly fitting the 2D velocity-delay maps such as those shown here. There are a growing number of real 2D RM datasets with the quality and cadence required to use this new method to recover transfer functions for real sources (G. A. Kriss et al. 2019; Y. Homayouni et al. 2023; Z.-X. Zhang et al. 2019; S. Wang et al. 2025), which we can then attempt to match with more physical models of the BLR to better understand its fundamental nature. This is a natural next step for this method and something we will demonstrate in future publications.

9. ACKNOWLEDGMENTS

This work was supported in part by NSF grants AST-1909711 and AST-2307983, and by an Alfred P. Sloan Research Fellowship (JD). All the *HST* data used in this work can be found on the MAST archive: 10.17909/t9-ky1s-j932 This work utilized the Alpine high performance computing resource at the University of Colorado Boulder. Alpine is jointly funded by the University of Colorado Boulder, the University of Colorado Anschutz, and Colorado State University and with support from NSF grants OAC-2201538 and OAC-2322260. KL is especially grateful to Sajal Gupta, Gisella de Rosa, and Matilde Signorini for many helpful discussions over the course of the project.

Software: Julia, including: `BroadLineRegions.jl`, `Flux.jl`, `CairoMakie.jl`, `Plots.jl`

REFERENCES

Akhaury, U., Starck, J.-L., Jablonka, P., Courbin, F., & Michalewicz, K. 2022, Frontiers in Astronomy and Space Sciences, 9, doi: 10.3389/fspas.2022.1001043

Blandford, R. D., & McKee, C. F. 1982, ApJ, 255, 419, doi: 10.1086/159843

Cackett, E. M., Bentz, M. C., & Kara, E. 2021, iScience, 24, 102557, doi: 10.1016/j.isci.2021.102557

Deesamutara, S., Chainakun, P., Worrakitpoonpon, T., et al. 2025, The Astrophysical Journal, 980, 257, doi: 10.3847/1538-4357/adae85

Glorot, X., & Bengio, Y. 2010, in Proceedings of Machine Learning Research, Vol. 9, Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, ed. Y. W. Teh & M. Titterington (Chia Laguna Resort, Sardinia, Italy: PMLR), 249–256. https://proceedings.mlr.press/v9/glorot10a.html

GRAVITY Collaboration, Abuter, R., Accardo, M., et al. 2019, The Messenger, 178, 20, doi: 10.18727/0722-6691/5166

GRAVITY+ Collaboration, :, Abuter, R., et al. 2023, arXiv e-prints, arXiv:2301.08071, doi: 10.48550/arXiv.2301.08071

He, K., Zhang, X., Ren, S., & Sun, J. 2015, CoRR, abs/1512.03385

Herbel, J., Kacprzak, T., Amara, A., Refregier, A., & Lucchi, A. 2018, Journal of Cosmology and Astroparticle Physics, 2018, 054, doi: 10.1088/1475-7516/2018/07/054

Homayouni, Y., De Rosa, G., Plesha, R., et al. 2023, ApJ, 948, 85, doi: 10.3847/1538-4357/acc45a

Horne, K. 1994, in Astronomical Society of the Pacific Conference Series, Vol. 69, Reverberation Mapping of the Broad-Line Region in Active Galactic Nuclei, ed. P. M. Gondhalekar, K. Horne, & B. M. Peterson, 23

Horne, K., Welsh, W. F., & Peterson, B. M. 1991, The Astrophysical Journal Letters, 367, L5, doi: 10.1086/185919

Horne, K., De Rosa, G., Peterson, B. M., et al. 2021, ApJ, 907, 76, doi: 10.3847/1538-4357/abce60

Innes, M. 2018, Journal of Open Source Software, doi: 10.21105/joss.00602

Innes, M., Saba, E., Fischer, K., et al. 2018, CoRR, abs/1811.01457. https://arxiv.org/abs/1811.01457

Kasliwal, V. P., Vogeley, M. S., & Richards, G. T. 2015, MNRAS, 451, 4328, doi: 10.1093/mnras/stv1230

Korista, K. T., Alloin, D., Barr, P., et al. 1995, ApJS, 97, 285, doi: 10.1086/192144

Kozłowski, S., Kochanek, C. S., Udalski, A., et al. 2010, ApJ, 708, 927, doi: 10.1088/0004-637X/708/2/927

Kriss, G. A., De Rosa, G., Ely, J., et al. 2019, ApJ, 881, 153, doi: 10.3847/1538-4357/ab3049

Krolik, J. H., & Done, C. 1995, ApJ, 440, 166, doi: 10.1086/175258

Li, D.-W., Hansen, A. L., Yuan, C., Bruschweiler-Li, L., & Brüschweiler, R. 2021, Nature Communications, 12, doi: 10.1038/s41467-021-25496-5

Liu, C., Sun, M., Dai, N., et al. 2022, GEOPHYSICS, 87, S249–S265, doi: 10.1190/geo2020-0904.1

Long, K., & Dexter, J. 2025, The Astrophysical Journal, 987, 196, doi: 10.3847/1538-4357/adda38

Long, K., Dexter, J., Cao, Y., et al. 2023, The Astrophysical Journal, 953, 184, doi: 10.3847/1538-4357/ace4bb

MacLeod, C. L., Ivezić, Ž., Kochanek, C. S., et al. 2010, ApJ, 721, 1014, doi: 10.1088/0004-637X/721/2/1014

Mandal, A. K., Woo, J.-H., Wang, S., et al. 2024, The Astrophysical Journal, 968, 59, doi: 10.3847/1538-4357/ad414d

Mangham, S. W., Knigge, C., Williams, P., et al. 2019, MNRAS, 488, 2780, doi: 10.1093/mnras/stz1713

Pancoast, A., Brewer, B. J., & Treu, T. 2014, MNRAS, 445, 3055, doi: 10.1093/mnras/stu1809

Peterson, B. M. 1993, PASP, 105, 247, doi: 10.1086/133140

Pozo Nuñez, F., Bañados, E., Panda, S., & Heidt, J. 2025, A&A, 700, L8, doi: 10.1051/0004-6361/202555421

Wang, S., Woo, J.-H., Barth, A. J., et al. 2025, The Astrophysical Journal, 983, 45, doi: 10.3847/1538-4357/adbca5

Welsh, W. F., & Horne, K. 1991, ApJ, 379, 586, doi: 10.1086/170530

Williams, P. R., Pancoast, A., Treu, T., et al. 2020, ApJ, 902, 74, doi: 10.3847/1538-4357/abbad7

Xu, L., Ren, J. S., Liu, C., & Jia, J. 2014, in Advances in Neural Information Processing Systems, ed. Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, & K. Weinberger, Vol. 27 (Curran Associates, Inc.). https://proceedings.neurips.cc/paper_files/paper/2014/file/1c1d4df596d01da60385f0bb17a4a9e0-Paper.pdf

Yu, W., Richards, G. T., Vogeley, M. S., Moreno, J., & Graham, M. J. 2022, The Astrophysical Journal, 936, 132, doi: 10.3847/1538-4357/ac8351

Zhang, Z.-X., Du, P., Smith, P. S., et al. 2019, ApJ, 876, 49, doi: 10.3847/1538-4357/ab1099

APPENDIX

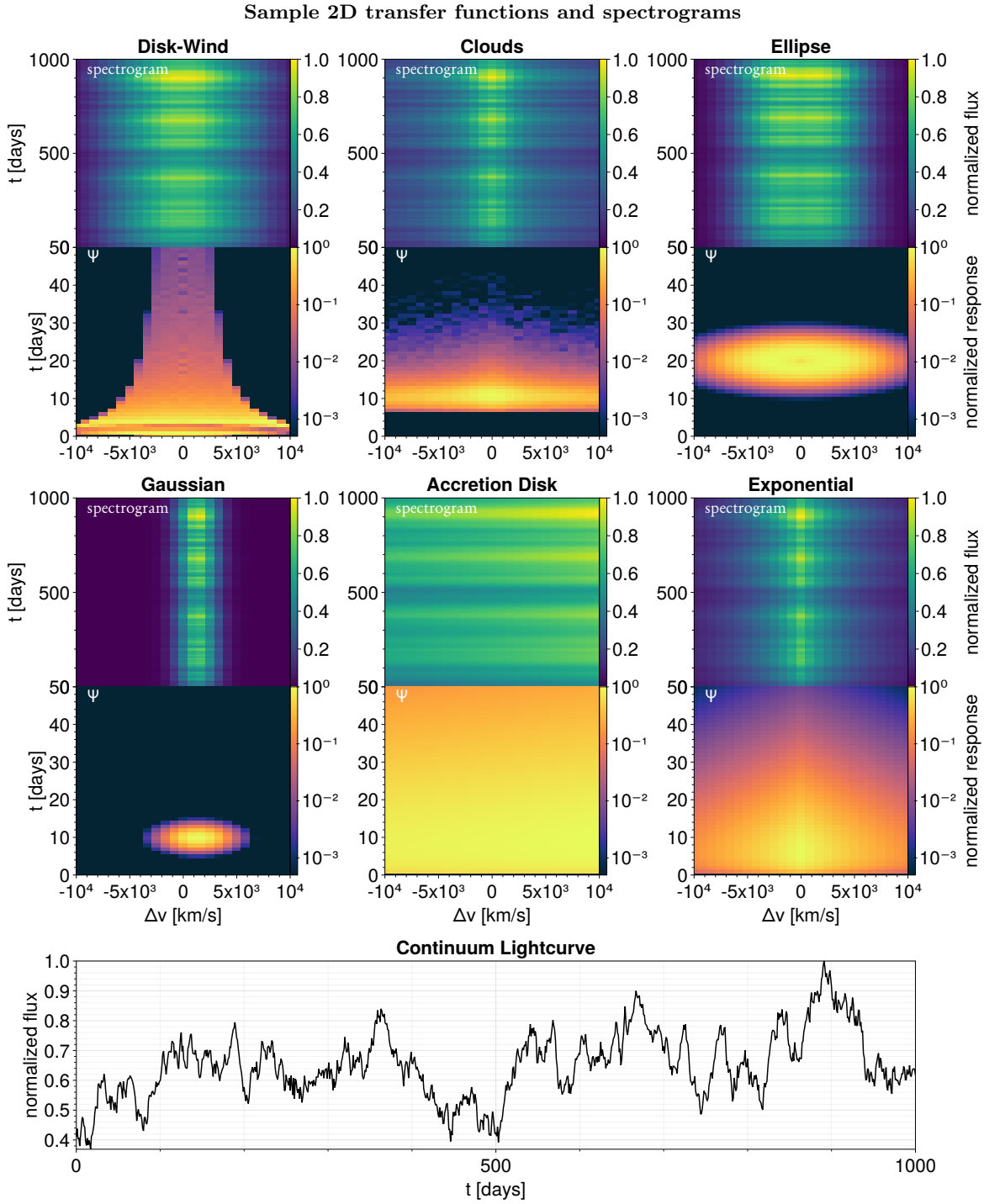


Figure 11. Similar to Figure 3, here we show representative samples from each class of "basis" transfer function used in training our models. **Top grid:** representative spectrograms and transfer functions for each basis function. The bottom panel of each grid entry shows the 2D transfer function and the top panel shows the spectrogram (2D visualization of all of the individual lightcurves) that results after convolving with the continuum and adding errors. The top panels thus represents sample inputs to the (D)CNN model and the bottom panel the expected outputs. **Bottom:** the continuum lightcurve used in these generating these examples.